

1 Quoodle Helper: A privacy-first, browser-based tool for
2 AI-assisted generation of multiple-choice questions from
3 teaching materials

4 **Gerrit Hirschfeld**  ¹

5 ¹ Bielefeld School of Business, HSBI University of Applied Sciences Bielefeld, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Submitted: 27 April 2026

Published: unpublished

License

Authors of papers retain copyright¹²
and release the work under a ¹³
Creative Commons Attribution 4.0⁴
International License ([CC BY 4.0](#))¹⁵

6 **Summary**

7 Quoodle Helper is a fully client-side web application that aids the conversion of existing
8 teaching materials — lecture slides, handouts, scripts, and notes in PDF, DOCX, PPTX,
9 TXT, or Markdown format — into multiple-choice quizzes. Text extraction runs entirely
10 in the browser; only the extracted text is forwarded to a large language model (LLM)
11 endpoint chosen and controlled by the educator. The resulting questions are exported as
an Excel or CSV file that can be edited offline and can be used with [Quoodle](#) (?), the
companion quiz-distribution tool for formative assessments. Together the two tools form a
lightweight, self-hosted authoring-and-delivery pipeline for knowledge checks that requires
no server infrastructure, no user accounts, and no third-party data processing beyond the
LLM endpoint selected by the user.

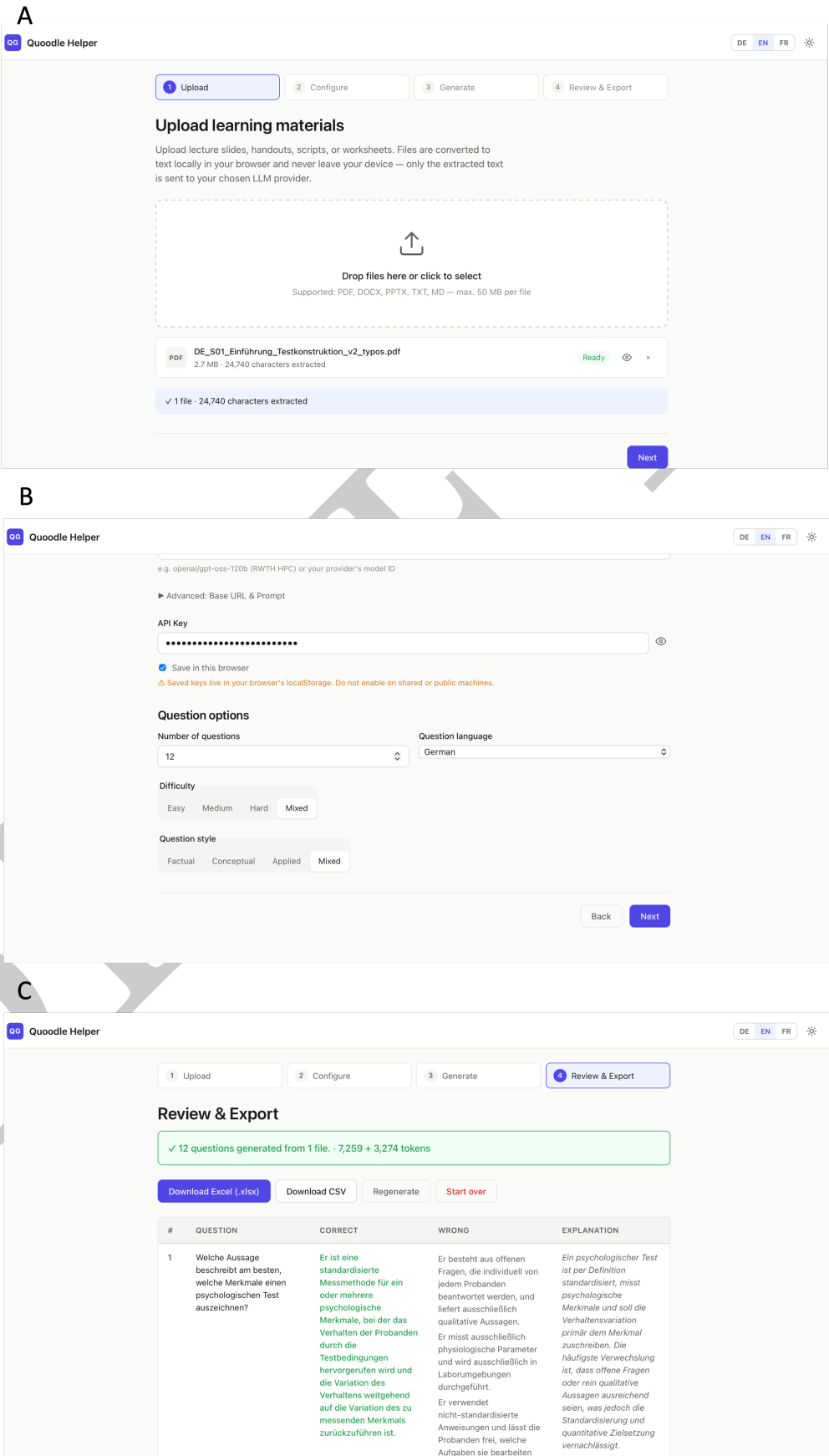


Figure 1: Screenshot from Quoodle-helper. Top upload screen, middle configurations screen with option to choose the number of items, language and difficulty, bottom resulting multiple-choice questions. 2

Hirschfeld. (2026). Quoodle Helper: A privacy-first, browser-based tool for AI-assisted generation of multiple-choice questions from learning materials. 2

Journal of Open Source Education, 1(1)ISSUE?, 364. <https://doi.org/10.xxxxx/draft>.

Statement of need

Short knowledge checks have well-documented learning benefits (Hattie & Timperley, 2007). The easiest way to implement these are multiple-choice items. But it is still a time-intensive task to prepare them. Constructing distractors that are plausible without being misleading, and explanations that address the most tempting wrong answer rather than merely restating the correct one, usually demands subject knowledge and substantial editorial effort (Haladyna et al., 2002). The practical consequence is that many educators rely on a small, recycled item pool, or forgo formative quizzes altogether.

Large language models have demonstrated the ability to produce exam-quality multiple-choice items from source texts and to match or exceed the discriminatory power of hand-written items in several subject domains (Biancini et al., 2024). Converting this capability into a tool that educators can actually adopt in the classroom, however, requires solving several further problems:

1. **Privacy and data protection.** Lecture slides and handouts can contain preliminary research findings, clinical vignettes, or examination blueprints that educators are not able or willing to upload to a commercial LLM, e.g. ChatGPT or Claude. EU institutions are additionally constrained by the General Data Protection Regulation and regional school-data-protection directives.
2. **Infrastructure and account friction.** Server-based tools require deployment, maintenance, and — almost universally — user accounts for both educator and learners.
3. **Vendor lock-in.** Tools tied to a single LLM provider inherit that provider's pricing, availability, and data-retention policies.
4. **Format incompatibility.** Generated items are typically delivered as formatted text that must be manually transferred into whichever quiz platform the educator uses, negating much of the time saving.

Quoodle Helper addresses each of these barriers. Because it is a static web application with no backend, it can be deployed on a university webspace, GitHub Pages, or a local development server with a single command. Because it accepts any OpenAI-compatible endpoint — including local inference servers such as Ollama and institutional HPC clusters — educators can choose a provider that matches their data-processing agreements. Because it exports directly to the spreadsheet format consumed by Quoodle (?), the path from source material to a live, feedback-rich quiz involves no manual reformatting.

Pedagogical design

The default item-generation prompt of Quoodle Helper is tuned for university-level exam items that assess conceptual understanding and transfer rather than text recall (Kratwohl, 2002). Specifically, the prompt instructs the model to:

- Avoid phrasing such as “according to the text” or “laut Vorlesung” (according to the lecture), which anchor items to surface memory of the source rather than understanding of the concept.
- Prefer questions about **relationships between concepts** over questions about isolated definitions.
- Construct distractors of comparable length and grammatical form to the correct answer, and — where possible — target a specific, nameable misconception.
- Write explanations that address the most tempting wrong answer, not merely restate the correct one.

This prompt design is aligned with established item-writing guidelines for higher-education assessment (Haladyna et al., 2002) and with the principle that elaborative feedback on

incorrect responses produces larger gains than correctness-only feedback (Shute, 2008). All columns generated and exported by Quoodle Helper correspond directly to the input columns expected by Quoodle, which displays the multiple choice test in a learning-friendly way.

Educators who work in specialised domains or who prefer a different assessment style can edit the system prompt through an **Advanced** panel in the configuration step. The edited prompt is stored locally in their browser so that this prompt can be re-used on subsequent visits; a single button reverts to the shipped template. This design gives educators direct control over item quality without requiring them to modify source code.

Difficulty and style are exposed as first-class parameters. The difficulty axis ranges from *easy* (factual recall) through *medium* (comprehension) to *hard* (analysis and evaluation), and the style axis covers *factual*, *conceptual*, and *applied* variants. Both parameters are interpolated into the system prompt at request time via named placeholders, so the model receives a single, coherent instruction set rather than a patchwork of conditional clauses.

Implementation

Quoodle Helper is a static single-directory application with no build step and no server-side runtime. It is structured as a four-step wizard (index.html) backed by six focused JavaScript modules:

- **extractors.js** — converts uploaded files to plain text entirely in the browser, using pdf.js for PDF, mammoth.js for DOCX, and JSZip for PPTX slide text and speaker notes.
- **providers.js** — a unified adapter that targets any OpenAI-compatible endpoint, the native Anthropic Claude API, and direct OpenAI access. URL normalisation (appending /chat/completions when the base URL ends in /v1) means users can paste either form of a provider address.
- **generate.js** — handles prompt assembly, source-text chunking for large documents, JSON parsing and schema validation of model responses, and near-duplicate deduplication via normalised string comparison.
- **export.js** — produces the output file via SheetJS: one worksheet named Questions, with a bold header row and one question per subsequent row, in the column order (Question, Correct Answer, Wrong 1, Wrong 2, Wrong 3, Explanation, Source) that Quoodle's upload parser expects.
- **i18n.js** — loads a flat JSON translation map for German or English and applies it to all data-i18n attributes in the DOM; the active language is persisted in localStorage and switchable at runtime without a page reload.
- **app.js** — orchestrates step navigation, event handling, and application state.

All four browser libraries on which the application depends (pdf.js 4.7.76, mammoth.js 1.9.0, JSZip 3.10.1, and SheetJS 0.18.5) are bundled under vendor/ and served from the server. The application makes no CDN calls, no analytics requests, and no calls to any endpoint other than the LLM provider the educator has explicitly configured. This ensures compliance with strict content-security policies and allows the tool to operate completely offline when paired with a local LLM-server such as Ollama.

API keys are stored in the browser storage (localStorage) with a visible warning displayed on the configuration screen. The repository ships with a German Impressum and Datenschutzerklärung (privacy policy) in accordance with § 5 DDG and the GDPR, with named placeholders indicating where a self-hosting educator should insert their own contact details.

The supported LLM providers and their defaults are listed in Table 1.

Table 1: Supported LLM providers and their default configuration.

Provider	Default base URL	Default model	API key required
Ollama	http://localhost:11434	llama3.1	no
Anthropic Claude	https://api.anthropic.com	claude-3.5	yes
OpenAI	https://api.openai.com	gpt-4o	yes
Local / OpenAI-compatible (default)	https://llm.hpc.itc.rwth-aachen.de/v1/	openai/gpt-oss-120b	yes

The source code is available at <https://github.com/gerhi/quoodle-helper>.

Acknowledgements

This project is a companion to Quoodle (?). Large parts of the codebase were drafted with the assistance of Anthropic’s Claude, under human design direction, review and verification.

References

Biancini, G., Ferrato, A., & Limongelli, C. (2024). Multiple-choice question generation using large language models: Methodology and educator insights. *Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization*, 584–590. <https://doi.org/10.1145/3631700.3665233>

Haladyna, T. M., Downing, S. M., & Rodriguez, M. C. (2002). A review of multiple-choice item-writing guidelines for classroom assessment. *Applied Measurement in Education*, 15(3), 309–333. https://doi.org/10.1207/s15324818ame1503_5

Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. <https://doi.org/10.3102/003465430298487>

Krathwohl, D. R. (2002). A revision of bloom’s taxonomy: An overview. *Theory Into Practice*, 41(4), 212–218. https://doi.org/10.1207/s15430421tip4104_2

Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1), 153–189. <https://doi.org/10.3102/0034654307313795>